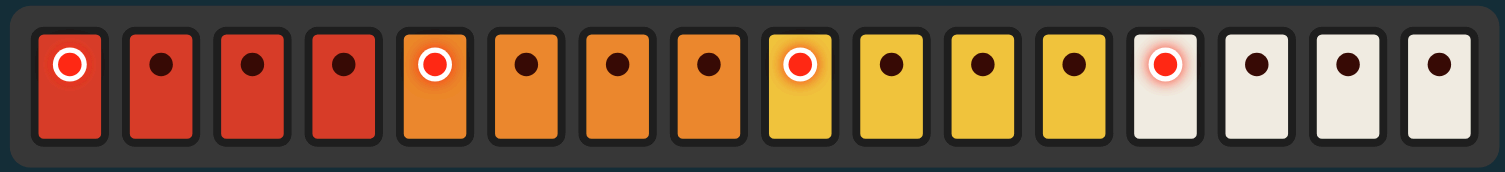
ACTAM 2026 · LAB 05

The drum machine as a class



model, controller, views, in Python

Last time, in the browser

A drum machine in the style of the TR-808, split in three parts.

part	what it is	in JavaScript
model	the data that rebuilds the machine	`keys`, `step`
controller	changes the model	a click flips `keys[i]`, `nextStep`
view	draws the model	`render`, `renderKey`

- Save and load became two lines: copy the model.
- `store = keys` did not save anything: it needed `keys.slice()`.

Today, in Python

A **class** keeps the model and the controller in one object.

part	today
model	`self.steps`, a list of booleans
controller	methods: `toggle`, `clear`, `mirror`
views	text, keys on the screen, sound

Then: copies, operators, JSON, and a drum machine of patterns.

Notebook · Setup Run the setup cell: you should see a row of keys and hear a kick.

A pattern from a string

```
kick_p = Pattern("x...x...x...x...")
kick_p.steps     # [True, False, False, False, True, ...]

Pattern("x..o")  # ValueError: a message of your own
```

- `"x"` is a hit and `"."` is a rest, as in lesson 04.
- The model is one attribute, `steps`: a list of booleans.

Notebook · A1 Write the constructor of Pattern.

Text, length, steps

```
kick_p.text()     # 'x...x...x...x...'
kick_p            # Pattern('x...x...x...x...')   __repr__
len(kick_p)       # 16                            __len__
kick_p[0]         # True                          __getitem__
```

- Before: `<__main__.Pattern at 0x10a2f...>`, and `kick_p[0]` fails.
- Each section writes the class again, with the methods so far.
- After writing a class again, make its objects again.

Keys on the screen

```
    def _repr_html_(self):
        return bar_html(self.steps)
```



- Jupyter calls `_repr_html_` for us, as Python calls `__repr__`.
- The colours are in `bar_html`, not in the model.

The sound view

```
def render(pattern, sound, step_s=0.125):
    ...

play(render(kick_p, kit["kick"]))
```

1. A step lasts `n = round(step_s * SR)` samples.
2. Start from zeros: `len(pattern) * n + len(sound)`.
3. At every hit `i`, add `sound` into the slice from `i * n`.

A beat

```
kick_p  = Pattern("x.......x.......")
snare_p = Pattern("....x.......x...")
hat_p   = Pattern("x.x.x.x.x.x.x.x.")

play(mix([render(kick_p, kit["kick"]),
          render(snare_p, kit["snare"]),
          render(hat_p, kit["hat"])])))
```

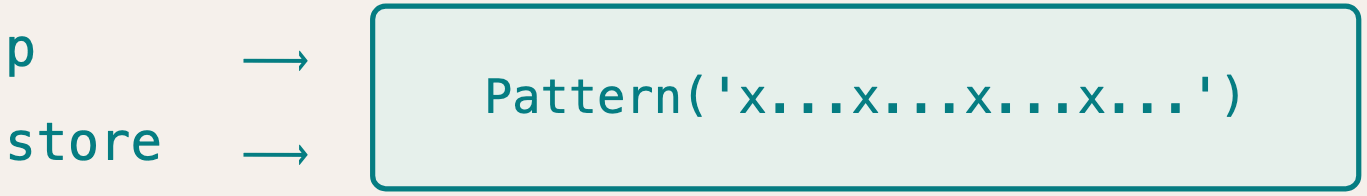
Billie Jean, from lesson 04. Now make a beat of your own.

Change the model

```
p = Pattern("xxxx............")
p.toggle(4)       # a click on key 4
p.mirror()        # backwards: [1, 2, 3][::-1] is [3, 2, 1]
p.clear()         # all off
```

- These methods change `self.steps` and give back nothing.
- `q = p.mirror()` makes `q` equal to `None`.
- Lesson 07's `a >> b` made a new clip instead. Both are good designs.

Equal, or the same?




```
store = p          # save?
p.toggle(2)        # store changes too
store is p         # True: one object, two names
```

Notebook · D1 Run the cells, and guess before running bad_copy.

A real copy, and `==`

```
store = p.copy()      # save
p.toggle(2)
p == store, p is store      # (False, False)

p = store.copy()      # load
p == store, p is store      # (True, False)
```

- `copy()`: a new pattern, with a list of its own.
- `__eq__` compares the steps. Without it, `==` is `is`.

Then, again, together

expression	meaning	method
`a >> b`	`a`, then `b`	`__rshift__`
`a * 3`	`a` three times	`__mul__`
`a | b`	a hit where either has one	`__or__`

- New patterns: `a` and `b` stay as they were.
- `|` on different lengths raises a `ValueError`.

Notebook · E1 Add >>, * and |.

Four bars

```
kicks = groove * 3 >> fill
```

Three bars of a groove and one with a fill, for each sound. Play them together.

Save the state, not the machine

```
text = kick_p.to_json()   # '{"steps": [true, false, ...]}'
again = Pattern.from_json(text)
again == kick_p, again is kick_p    # (True, False)
```

- `json.dumps` and `json.loads`: data to text and back.
- `@classmethod`: a method of the class, called with `cls`. Another way to build an object.

G · EXTENSIONS

For whoever is ahead

A drum machine of patterns

```
dm = DrumMachine({"kick":  Pattern("x.......x.x....."),
                  "snare": Pattern("....x.......x..."),
                  "hat":   Pattern("x.x.x.x.x.x.x.x.")})
dm.tick()         # step = (step + 1) % length
dm                # three rows of keys, the step in green
play(dm.render())
```

- The model: `tracks`, `step_s`, `step`. An object made of objects: **composition**.

Euclidean rhythms, and more

- `euclid(k, n)`: step `i` is a hit when `(i * k) % n < k`.
- `euclid(3, 8)` is `x..x..x.`: the *tresillo*.
- Three against four: `Pattern("x..") * 4` and `Pattern("x...") * 3`.
- Your own keys: `my_bar_html(steps)`, with inline CSS.
- A `@dataclass` pattern: which methods does it write for us?

What we built

model	`Pattern.steps`, `DrumMachine.step`
controller	`toggle`, `clear`, `mirror`, `tick`
views	`__repr__`, `_repr_html_`, `render`
copies	`copy`, `==` and `is`
operators	`>>`, `*`, `|`
saving	`to_json`, `Pattern.from_json`

A class keeps the data and what changes it together. The views look at it from outside.